

CSCE 611 OPERATING SYSTEMS · MODULE 12 · LECTURE NOTES

Distributed Aspects: RPC and Network File Systems

Where the operating system's abstractions stop working.

MODULE OUTCOMES

- Explain RPC and why it cannot be transparent.
- State the fallacies of distributed computing and their consequences.
- Explain at-most-once, at-least-once, and exactly-once semantics.
- Explain NFS's stateless design and what it buys and costs.
- Explain caching and consistency trade-offs in distributed file systems.

1 Remote procedure call

RPC's premise is that a call to a remote service should look like an ordinary function call. A client **stub** marshals the arguments into a message, sends it, and blocks; a server stub unmarshals, invokes the real function, and returns the result the same way.

The appeal is obvious: distribution becomes an implementation detail. The difficulty is that distribution is not an implementation detail, and an abstraction that pretends otherwise leads programmers to write code that cannot handle the failures that will occur.

Local call	Remote call
Takes nanoseconds.	Takes microseconds to seconds — six orders of magnitude more, and highly variable.
Cannot fail without the caller also failing.	Can fail independently. The callee may die while the caller continues.
Pointers are valid.	Pointers are meaningless; everything must be serialised and copied.
Shares memory with the callee.	Shares nothing.
Failure means the whole process crashed.	Failure may mean 'no answer yet' — an outcome with no local analogue.

YOU CANNOT DISTINGUISH SLOW FROM DEAD

A request is sent. No reply arrives within the timeout. What happened?

The request may have been lost in transit. It may have arrived and the server is merely slow. It may have executed successfully and the *reply* was lost. The server may have crashed before executing it, or after.

From the client these are indistinguishable. No choice of timeout resolves the ambiguity, because any timeout is potentially too short for a server that is heavily loaded rather than dead.

Essentially every difficulty in distributed systems descends from this single fact: it is why consensus algorithms are subtle, why exactly-once delivery is impossible, why failure detectors are necessarily imperfect, and why retries must be safe to repeat.

2 Delivery semantics

Semantics	Guarantee	How to obtain it
At-most-once	The operation never executes more than once. It may not execute at all.	Do not retry. Suitable when a duplicate is worse than a loss.
At-least-once	The operation executes, possibly several times.	Retry until acknowledged. Requires the operation to be idempotent , or duplicates cause damage.
Exactly-once	The operation executes precisely once.	Not achievable at the transport layer over an unreliable network.

WHAT 'EXACTLY-ONCE' SYSTEMS ACTUALLY DO

Exactly-once delivery is impossible, and this is a theorem rather than a gap in current engineering: the sender cannot know whether a lost acknowledgement means the operation happened.

Systems that advertise exactly-once semantics achieve it differently: they retry freely, giving at-least-once delivery, and make the **effect** idempotent — typically by attaching a unique identifier to each operation and having the receiver discard duplicates.

The design lesson is concrete. `transfer(account, +$100)` is not idempotent, and a retry doubles the transfer. `set_balance(account, $500)` is idempotent. `apply(txn_id=abc123, +$100)` with server-side duplicate detection is idempotent and preserves the semantics you wanted.

Make the operation safely repeatable and the delivery problem stops mattering. This single technique resolves a large fraction of distributed systems difficulty.

2.1 The fallacies of distributed computing

Peter Deutsch and James Gosling's list of assumptions that newcomers to distributed systems make, all of which are false and all of which a local function call may safely make:

- 1 The network is reliable.

- 2 Latency is zero.
- 3 Bandwidth is infinite.
- 4 The network is secure.
- 5 Topology does not change.
- 6 There is one administrator.
- 7 Transport cost is zero.
- 8 The network is homogeneous.

The list is thirty years old and has not aged. Note the relationship to §1: these are precisely the assumptions that RPC's transparency invites, which is the strongest argument against making remote calls look local.

3 Network file systems

3.1 NFS and statelessness

NFS's defining decision is that the server keeps **no per-client state**. There is no server-side notion of an open file; each request carries a file handle and an explicit offset and is completely self-contained.

THE FAILURE MODEL DROVE THE DESIGN

With no per-client state, server crash recovery is trivial: the server reboots and begins answering requests again. Clients, which were retrying anyway, simply succeed on a later attempt and may not notice the outage at all.

A stateful server would have to reconstruct every client's open files, offsets, and locks after a crash — and would have to distinguish a client that had crashed from one that was merely slow, which §1 says it cannot.

The price is paid in semantics. Every operation must be idempotent, because clients retry. There is no server-side locking in the base protocol. And the Unix behaviour of deleting an open file (Module 08) cannot work, because the server does not know the file is open — NFS clients fake it by renaming the file to a hidden name, which is why stray `.nfs*` files appear.

3.2 Caching and consistency

CACHING FORCES A CHOICE YOU CANNOT AVOID

Without client-side caching, every read is a network round trip. The result is correct and unusably slow.

With caching, a client can read data that another client has already changed. Correctness has been traded for speed, and the only question is how much.

NFS chose close-to-open consistency: a client that opens a file after another client closed it sees that client's writes. Within an open session, no guarantee. This is substantially weaker than local file semantics and sufficient for the dominant use case of one writer at a time.

AFS chose callbacks: the server records which clients hold cached copies and notifies them when a copy becomes invalid. The consistency is much stronger — and the server is now stateful, which is precisely what NFS avoided, so crash recovery requires rebuilding the callback table.

There is no design that is simultaneously fast, strongly consistent, and simple to recover. This is the CAP trade-off appearing in a small and concrete setting.

	NFS	AFS
Server state	None.	Tracks which clients cache which files.
Consistency	Close-to-open.	Callbacks — near-local.
Crash recovery	Trivial — just restart.	Must rebuild callback state; clients must revalidate.

	NFS	AFS
Caching granularity	Blocks.	Whole files, cached to local disk — designed for a campus-scale network with high latency.
Scalability	Good.	Better — whole-file caching means far fewer round trips.

4 What generalises

- 1 **Make operations idempotent.** Then retrying is always safe, and the impossibility of exactly-once delivery stops being a problem. This is the single most valuable technique in the module.
- 2 **Do not hide the network.** An abstraction that makes remote calls indistinguishable from local ones encourages every fallacy in §2.1. Make latency and failure visible in the interface — which is why modern RPC frameworks expose deadlines, retries, and explicit error types.
- 3 **Choose a consistency model deliberately and document it.** Every distributed system has one whether or not anyone wrote it down, and an undocumented one is discovered by users.
- 4 **Treat partial failure as normal.** In a distributed system, some component is always down. Code that handles this only as an exceptional path will spend most of its life on that path.

CSCE 662 and CSCE 678 develop all of this properly — consensus, replication, consistency models, and the impossibility results that bound what is achievable. This module's purpose is to show where the operating system's abstractions stop working, and why.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
OSTEP — Chapters 48–50 (Distributed Systems, NFS, AFS)	RPC, NFS's statelessness, and the AFS comparison, written clearly.
MIT 6.5840 Distributed Systems (free lectures and labs)	The natural follow-on. Build Raft and a fault-tolerant key-value store.
Waldo et al. — A Note on Distributed Computing (free)	The classic argument that local and remote objects cannot be unified. Short, and it settled the question.
Deutsch & Gosling — The Eight Fallacies of Distributed Computing	The original list, with commentary.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement a simple RPC mechanism over TCP: marshal arguments, send, block, unmarshal the reply. Then add a timeout and decide what to do when it expires.
- 2 Demonstrate the slow-versus-dead problem: make the server sleep longer than the client's timeout and observe the client retrying an operation that is still executing.
- 3 Implement at-least-once semantics with retries. Use a non-idempotent operation and demonstrate the duplicate effect.
- 4 Add a deduplication key and server-side duplicate detection. Demonstrate that retries are now harmless.
- 5 Set up an NFS mount between two machines or VMs. Write from one and read from the other, and determine experimentally when changes become visible.
- 6 Delete a file on an NFS mount while a process on another client has it open. Observe the `.nfs*` file and explain it.
- 7 Measure the cost of close-to-open consistency: time a workload that opens, reads, and closes repeatedly against one that holds the file open.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Give four ways a remote call differs from a local one.
- 2 Why can a client not distinguish a slow server from a dead one, and what follows from that?
- 3 Define at-most-once, at-least-once, and exactly-once, and say which is impossible.
- 4 How do systems that advertise exactly-once semantics actually work?
- 5 Why is the NFS server stateless, and what three things does that cost?
- 6 What consistency model does NFS provide, and how does AFS differ?
- 7 Name four of the eight fallacies and say why RPC transparency invites them.